BLEND: A Unified Data Discovery System

1st Mahdi Esmailoghli
TU Berlin & BIFOLD
Berlin, Germany
esmailoghli@tu-berlin.de

2nd Christoph Schnell
Leibniz Universität Hannover
Hannover, Germany
schnell@dbs.uni-hannover.de

3rd Renée J. Miller
University of Waterloo
Waterloo, Canada
rjmiller@uwaterloo.ca

4th Ziawasch Abedjan
TU Berlin & BIFOLD
Berlin, Germany
abedjan@tu-berlin.de

2025 IEEE 41st International Conference on Data Engineering (ICDE) | 979-8-3315-3603-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDE65448.2025.00061

***Abstract*—Most research on data discovery has so far focused on improving individual discovery operators such as join, correlation, or union discovery. However, in practice, a combination of these techniques and their corresponding indexes may be necessary to support arbitrary discovery tasks. We propose BLEND, a comprehensive data discovery system that supports existing operators and enables their flexible pipelining. BLEND is based on a set of lower-level operators that serve as fundamental building blocks for more complex and sophisticated user tasks. To reduce the execution runtime of discovery pipelines, we propose a unified index structure and a rule- and cost-based optimizer that rewrites SQL statements into low-level operators when possible. We show the superior flexibility and efficiency of our system compared to ad-hoc discovery pipelines and stand-alone solutions.**

I. Introduction

The potential for large structured data lakes, such as governmental data lakes [19], [61], enterprise-level corpora [10], [15], and public-access lakes [22], [14], [33], has spurred various avenues of research, all aimed at empowering data scientists with the ability to extract further insights for their diverse tasks, such as machine learning (ML) [23], exploratory data analysis [41], and data cleaning [40]. To support these tasks, various data discovery operatorion have been studied, such as **join discovery** [24], [69], [63], [62], [59], [27], [70], [20], [16], [56], [50], **union search** [45], [11], [36], [13], [25], [34], and **correlation discovery** [23], [49], [9], [17].

Existing research has focused on each operator individually without considering possible opportunities for combining, pipelining, and optimizing them. Yet, fulfilling a real-world data discovery task often requires a combined application of these operators. For instance, in the case of data enrichment for ML, the user needs to use a combination of join, union, and correlation discovery [9], [45]. To achieve this, we need a unified data discovery system.

Requirements for a unified discovery system. A trivial approach to building such a unified system is to federate all indexes and algorithms via a common interface. Even after overcoming the inherent software and interface heterogeneity among prototypes, the execution of complex data discovery tasks that combines these operators across systems is inefficient and inflexible in terms of both storage and maintenance. This stems from several key challenges:

Lack of Flexibility. Introducing new criteria or discovery operators is challenging since new indexes and algorithms must be integrated into an already complex system.

Storage and Maintenance Overhead. Multiple independently developed index structures must be generated and stored. The aggregated system will comprise an incoherent set of potentially redundant structures needing maintenance.

Lack of Optimization. The federation of independent operators under the same abstraction does not lend itself to optimization. To showcase the operational complexity of a rather simple discovery task, consider following example:

Example 1. Fig. 1 illustrates a user table S along with three tables $T1$, $T2$, and $T3$ from a data lake. S contains information about departments in a company and the name of their corresponding leaders, i.e.,"Head". The user wants to fill in missing values of S but already knows that the head of the IT department, "Tom Riddle", has left the company. Therefore, any table that contains the projection ("IT","Tom Riddle") is outdated. Thus, the user defines the data discovery for missing values as follows: *Find top-*1 *table that contains ("HR" and "Firenze" in a row and "HR", "Marketing", "Finance", "IT", ... in a column but does not contain rows with "IT" and "Tom Riddle").*

To solve this task with existing discovery operators, one could use single-column join discovery, such as Josie [69], to look up overlapping column values, and multi-column join discovery, such as MATE [24], to find overlapping column combination values. With MATE, one could find tables containing the projection ("HR", "Firenze") as result set rs_1. Also, MATE can be used to identify tables containing the tuple ("IT", "Tom Riddle") as result set rs_2. Josie can be employed to find tables that join only on the department values "HR", "Marketing", "Finance", ... as result set rs_3. Finally, the equation $((rs_1 \cap rs_3) - rs_2)$ is the answer to the user's query. According to Fig. 1, $rs_1 = \{T2, T3\}$, $rs_2 = \{T2\}$, and $rs_3 = \{T1, T2, T3\}$. Thus, the answer to the discovery query is $\{T3\}$, which contains amendable values for the "Head" of the department column in S. •

The example illustrates that running independent discovery redundant and inefficient. For each look-up, combinations of the three tables $T1$, $T2$, and $T3$ may be processed. However, knowing that $T1$ and $T2$ are not desired tables, we could exclude them from follow-up operators: first, by checking all three tables to find those containing ("HR" and "Firenze"); then, by focusing only on $T3$ and $T2$ and dropping the outdated $T2$; and finally, by verifying $T3$ for overlapping department names. Intuitively, we could obtain the same results by resorting to application code. We aim to

2375-026X/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDE65448.2025.00061

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:53 UTC from IEEE Xplore. Restrictions apply.



Fig. 1: Discovering up-to-date tables to fill in the NULLs.

achieve the same flexibility within a discovery system with a unified language to optimize custom discovery processes. Such abstraction becomes necessary in case of complex queries with a variety of operators and result sets with millions of tables.

Contributions We introduce BLEND, a unified system that enables users to create complex discovery plans. With no need for compiling, installing, or adapting additional discovery systems, BLEND streamlines the discovery process. Given a data discovery plan, a data lake, and a parameter k, BLEND retrieves the top-k tables from the lake that are most relevant to the defined plan. Our key contributions are as follows:

1) **Introduction of an abstraction layer:** We design a novel abstraction that unifies the building blocks of different discovery tasks. We propose a set of atomic search operators, which users can declaratively compose to efficiently discover relevant tables. These building-blocks are implemented using primitive database functions, allowing the majority of computations to run within the database.
2) **Optimized execution of tasks:** We develop a two-phase plan optimizer that combines rule- and cost-based query optimization techniques. It re-orders the execution of operators based on predefined rules, ensuring efficient execution while preserving the intended logic of the discovery task. Furthermore, BLEND optimizes individual operators by rewriting corresponding queries using intermediate results.
3) **Integrated index:** We studied fifty-three index structures from thirty data discovery papers and propose a unified index that efficiently supports the execution of the introduced operators and their compositions. The index is designed as a single database table, enabling the system to additionally benefit from the database-level query optimizations.
4) **Simplification of complex tasks using SQL operations:** Our proposed index structure allows for the translation of BLEND's operators into SQL queries. Even complex tasks, such as correlation discovery, are reduced to SQL-based operations, eliminating the need for custom algorithms or external implementations. By integrating multiple operators into declarative pipelines, BLEND minimizes user effort and simplifies the creation of discovery tasks.
5) **User study:** We conducted a user study with data experts to understand their discovery preferences and evaluate the effectiveness of BLEND in addressing these needs.

II. Problem Statement

Given the requirements formulated in the introduction, we formalize the following problem statement. Let $\mathcal{D} = \{T_1, T_2, \ldots, T_n\}$ be a data lake, where each T_i is a table. Given a query table S with insufficient data and a discovery plan P by the user, the goal of BLEND is to enable the formulation of the plan and its optimized compilation for execution. The result of the execution is the subset $\mathcal{T}' \subseteq \mathcal{D}$ that enriches S. With this functionality in mind, we strive to solve four technical problems (**Pr.1-4**):.

First, the language for defining a plan should support the formulation of the most prominent discovery operations from the literature (**Pr.1**):

- **Keyword search**: discovered tables must overlap with a given set of keywords.
- **Join discovery**: discovered tables must be joinable with S on one or more columns.
- **Union discovery**: rows in discovered tables must be unionable with rows in S.
- **Correlation discovery**: a column in each discovered table must correlate with the user-defined numerical target in S.

Second, the system should also support any set-based aggregation of these operation's results (**Pr.2**).

Third, we require an index that serves all of the possible discovery operations but is leaner than the ensemble of the required individual indexes. More formally, assuming that an independent system exists for each of the aforementioned discovery operations M_{op_i}, where $1 < i < n$, the storage required for our index to execute any combination of operators in OP must be smaller than $\sum_{i=1}^{n} \text{Storage}(M_{op_i})$, where $Storage(M_{op_i})$ is the storage size required for the state-of-the-art index in M_{op_i} (**Pr.3**).

Finally, the system should satisfy two runtime constraints. The implementation of each (M_{op_i}) in the unified system should yield a comparable runtime while any aggregation and combination of more than two M_{op_i} should yield superior runtime compared to their execution as concatenated black box discovery system. Formally, assuming a discovery plan P is an arbitrary combination of operators in OP, i.e, $P \in OP^*$, the execution runtime of P in BLEND should satisfy

$$\text{Time}(P_{\text{BLEND}}) \leq \sum_{j|op_j \in P} \text{Time}(M_{op_j}).$$

Satisfying the previous constraint should not be a burden on how the user formulates the plan. BLEND must identify a good (near optimal) ordering of the operations based on the expected runtime and intermediate result size (**Pr.4**)

III. System Overview

Fig. 2 depicts an overview of BLEND's architecture depicting its online and offline phases. The online phase involves four stages: task definition, plan formulation, plan optimization, and execution.

First, the user defines their data discovery task using BLEND's API (Fig. 2a). Here, the plan *find_dep_heads* is

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:53 UTC from IEEE Xplore. Restrictions apply.

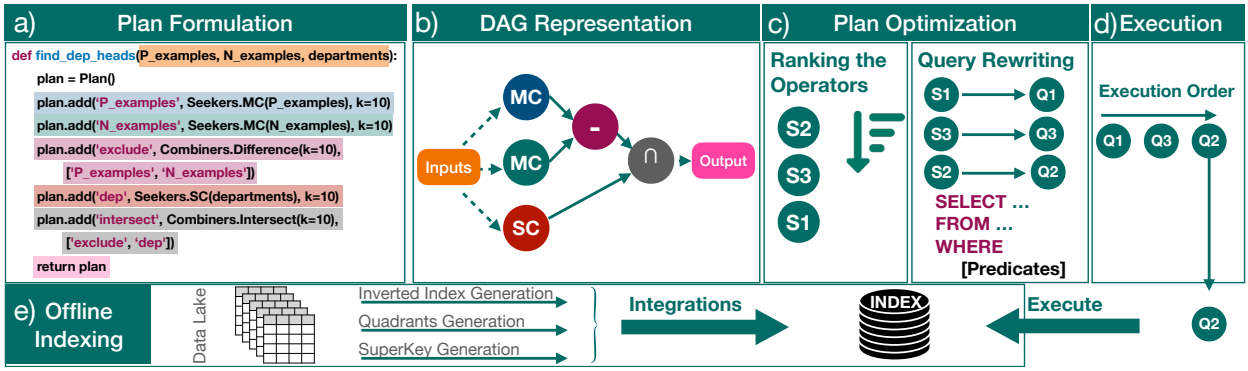


Fig. 2: BLEND's architecture.

written for the use case of Fig. 1. It finds tables containing a set of positive examples and the given department names but not negative examples. The main primitives in BLEND's API are *Plan*, *Seekers*, and *Combiners*. The latter two, *Seekers* and *Combiners*, are the lower-level operators in BLEND, which are implemented in SQL. A discovery task is created by instantiating a *Plan* object, to which one can add *Seekers* and *Combiners*. Seekers are added by specifying an identifier string and parameters, such as input lists and top-k. BLEND transforms the plan into a directed acyclic graph of operators, where edges represent the data flow (Fig. 2b). Once the discovery plan is written and the DAG is generated, BLEND leverages a combination of rule-based and cost-based optimization to re-rank and rewrite operators for faster execution (Fig. 2c). The low-level operators either resemble or contain SQL queries on BLEND's index which is organized as a fact table. Based on the cost model, BLEND rewrites the SQL statements by injecting predicate placeholders enabling the system to take advantage of cardinality estimates of the intermediate results. Once the queries are generated, they are sent to the execution engine to run in the database (Fig. 2d). In the offline phase, BLEND builds the necessary index and data structures (Fig. 2e). The index comprises and unifies three structures: inverted index [5], *super keys* [24], and quadrants [49].

IV. OPERATORS

BLEND's operators are designed to be composable, enabling complex plans, and simple for efficient execution. BLEND leverages two sets of operators: seekers and combiners.

A. Seeker operators

A seeker receives a set of columns, Q, as the input and returns the top-k most relevant tables. k is a user-defined parameter. We propose four seekers: *single-column*, *keyword*, *multi-column*, and *correlation* seekers.

1) Single-Column (SC) Seeker: The SC seeker receives an input column Q, typically from an input (query) table and returns a list of k tables. These tables contain a column that overlaps the most with Q (sorted by number of overlaps).

2) Keyword (KW) Seeker: The KW seeker returns the top-k overlapping tables based on a given set of keywords. The main difference between the KW and SC seekers is that the KW seeker measures the overlap throughout entire candidate tables rather than single columns. Returned tables are sorted based on the number of overlaps.

3) Multi-Column (MC) Seeker: The MC seeker receives Q that has multiple columns and discovers the top-k tables in the data lake that the most with the tuples in Q. The returned tables are sorted based on the overlap.

4) Correlation (C) Seeker: This seeker receives two columns Q, $Q = \{Q_j, R\}$, where Q_j is a join key and R is a numerical target column. The correlation seeker returns the top-k tables that join on Q_j and contain a column that correlates the most with R, sorted by the correlation coefficient.

B. Combiner operators

Combiners enable the discovery task specific composition of seeker results. Combiners receive a set of table collections, where each collection is the result of a seeker or another combiner. A combiner merges the table collections according to its defined set operation and returns a new set of tables. We implemented the most common set operators in BLEND: *Intersection* ($\cap$), *Union* ($\cup$), *Difference*($\setminus$), and *Counter*. Note that the user can introduce new combiners to the system. The **Intersection** and **Union** combiner receive the output of two or more seekers and returns the intersection/union of the tables, respectively. The **Difference** combiner receives only two sets of tables and returns the tables that exist only in the first table set. The **Counter** combiner receives multiple sets of tables, counts the occurrence of the table identifiers, and returns a list of tables sorted in descending order based on their frequencies.

C. Discovery Language Grammar

With the given seekers and combiners, the grammar of the supported discovery plans in BLEND can be defined as follows:

```
expression ::= seeker(Q) |
combiner(expression(,expression)+)
seeker ::= KW | SC | MC | C
combiner ::= ∩ | ∪ | \ | Counter
Q ::= keyword | table
```

Any `expression` based on the grammar above can be subject to optimization of BLEND. Further, it is possible to

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:53 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Mapping between Indexes and data discovery tasks.

Task	Keyword Search	Join Search	MC Join Search	Union Search	Correlation Search
Literature References	[51], [66] [44], [26] [5], [13], [12]	[5], [16], [62], [26], [10] [56], [20], [68], [50], [8] [70], [63], [24], [69] [59], [27], [51], [13]	[24]	[67], [45], [36] [25], [68], [35] [5], [51], [6] [8], [13], [11]	[23], [49] [48]

CellValue (nvarchar)	TableId (integer)	ColumnId (integer)	RowId (integer)	SuperKey (byte)	Quadrant (bool)
...	...	...	...	...	...

Fig. 3: The lake index is a single relational table: *AllTables*.

combine multiple of such expression via native python control flows, each being optimized independently.

V. Index

Each common discovery task requires its customized index structure. Using all of them together, has several drawbacks with regard to data consistency, application abstraction, and index maintenance. Thus, we make two contributions by first identifying a concise set of necessary indexes and then integrating them within a unified layout:

I. Identifying relevant indexes. We studied fifty-three index structures proposed in 30 data discovery articles, as illustrated in Table I. Each proposed index addresses a specific data discovery task. Subsequently, we carefully chose three index structures that cover all common discovery tasks and are compatible with each other, then adapted them for BLEND:

1) To enable content-to-table lookups, which cover keyword, join, and some union search tasks, we employ ideas from the DataXFormer system [5]. It introduced an inverted index, which stores cell values and their corresponding location, i.e., table, column, and row identifiers within a single structure.
2) MATE [24] proposes the XASH index. This hash-based structure enables fast multi-column join discovery.
3) The QCR index [49] calculates the Quadrant Count Ratio statistic (QCR) that approximates linear correlation. Alternatively, one could use *Order Index* proposed in the COCOA paper [23], however, this index is tailored for exact and nonlinear correlations as opposed to the QCR index, which makes the *Order Index* less efficient. We modify the QCR index in a way that enables in-DB calculation of correlation estimation via SQL (see Section IV-A).

II. Designing the layout. To provide a unified interface for operators on top of the aforementioned indexes, we propose a simple layout that is easy to maintain, prevents data redundancy, and is storage efficient. BLEND serializes these indexes into a database relation called `AllTables` as shown in Fig. 3. Implementing the index as a relation enables BLEND to benefit from in-DB features, e.g., query optimizations.

`AllTables` includes four columns from the DataXFormer index [5]: *CellValue*, *TableId*, *ColumnId*, and *RowId*, mapping each value to its locations in the lake. To facilitate fast value lookup and table loading, we create two in-database indexes on the *CellValue* and the *TableId* columns, respectively. An additional column stores the XASH index, i.e., a *super key*, for the given table and row [24]. The *super key* aggregates row values into a single hash, enabling efficient discovery of complex multi-column matches.

The original QCR index is not directly compatible with `AllTables` as it requires the enumeration of column pairs. Santos et al. [49] use the QCR [31] statistic to estimate the Pearson correlation between candidate columns and a target. To approximate the correlation between two sets of values X and Y, QCR divides the observations into four quadrants based on the relative position of the observations compared to their corresponding set averages. If a pair of values, x_i and y_i, are both greater or both smaller than their corresponding averages, they belong to Quadrant I or III, respectively. Otherwise, they are placed in either Quadrant II or IV. The QCR is computed as $QCR = \frac{n_I + n_{III} - n_{II} - n_{IV}}{N}$, where $n_{Quadrant}$ is the number of observations in the quadrant, and N is the number of all observations. for any two column pairs in a table, where one is categorical and one numerical, the baseline approach hashes each row based on the categorical value and the quadrant entry of the numerical value. The smallest h hashes are stored inside the index. During retrieval, the same hash is generated for the join and the target columns in Q and the minimal values are matched against the entries in the index.

Using our index structure, we calculate the correlation inside the database avoiding the need for further data loading and serialization to the application level. To align the original index with our design, we propose a novel representation that only requires one additional `Quadrant` column to store a Boolean per cell. *Quadrant* is set to `1` if the cell value is larger than or equal to the column average, and `0` otherwise. For Non-numerical data points *Quadrant* is set to *Null*. In contrast to the baseline, we invoke the sample size h while querying. With this approach, we do not constrain the comparison to the smallest h hash values but simply sample h hash values at random. This has the benefit that the hash size can be dynamically chosen without re-indexing the lake.

VI. Seeker Implementations

We now discuss operator implementations by detailing how operators use the index to efficiently return the top-k tables.

SC Seeker. Listing 1 shows the implementation of the SC seeker as an SQL query on top of *AllTables*. The query leverages the `WHERE` clause (Lines 1-2) to find the overlapping columns and returns the top-k tables (Lines 3-5) sorted based on the column in descending order (Line 4).

```
1 SELECT TableId FROM AllTables
2 WHERE CellValue IN (Q)
3 GROUP BY TableId, ColumnId
4 ORDER BY COUNT(DISTINCT CellValue) DESC
5 LIMIT K;
```

Listing 1: SQL implementation of the SC seeker.

KW Seeker. A standard adaptation of the SC seeker could directly be used for standard keyword search, where the `ColumnId` can be skipped in the `GROUP-BY` clause. For brevity, we skip the corresponding SQL code.

MC Seeker. The MC seeker requires not only identifying tables containing values from all input columns but also

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:53 UTC from IEEE Xplore. Restrictions apply.

ensuring consistent value alignment. This means that the values from the same row in Q must occur in the same row in the candidate tables. MATE [24] ensures this value alignment by representing table rows via an aggregated hash, i.e., *super key*, which serves as a bloom filter to prune non-joinable rows without checking the individual values and their alignment.

First, MATE uses an SQL query to only detect candidate table rows with any arbitrary combination of values from Q. The SQL query is shown in Listing 2. Note that the listing only shows the generated query for two columns but BLEND can generate the query for any number of columns.

```
SELECT * FROM
    (SELECT * FROM AllTables
    WHERE CellValue IN (Q1)) AS Q1_index_hits
    INNER JOIN
    (SELECT * FROM AllTables
    WHERE CellValue IN (Q2)) AS Q2_index_hits
    ON Q1_index_hits.TableId = Q2_index_hits.TableId
    AND Q1_index_hits.RowId = Q2_index_hits.RowId
```

Listing 2: The first phase of the MC seeker.

The query fetches index rows, including hash values, where the values from mutually exclusive columns of the same table appear in the same set of candidate table rows. The exact alignment of actual tuples is validated on the application level. Then, it uses the *super key* of each row to filter rows with wrong alignment of value combinations from Q and validates the remaining rows through exact match validation.

```
SELECT Keys.TableId FROM
(SELECT * FROM AllTables WHERE rowid < h AND
    C_Values IN ($Q_j$)) keys
INNER JOIN
(SELECT * FROM AllTables WHERE rowid < h AND
    Quadrant is not NULL) nums
ON keys.TableId = nums.TableId AND keys.RowId = nums
    .RowId
GROUP BY keys.TableId, nums.ColumnId, keys.ColumnId
ORDER BY ABS($SCORE$) DESC
LIMIT k;
```

Listing 3: SQL implementation of the correlation seeker.

Correlation Seeker. Listing 3 shows the implementation of the correlation seeker. First, the query fetches value pairs, where the first value is from Q_j and the second one is a value from a numerical column (Lines 1-5). This is done through an `INNER JOIN`. To ensure that the second value is numerical, we checks if *Quadrant* is non-NULL. Both `WHERE` clauses in the sub-queries (Line 4) sample h rows for correlation calculation as described in Section V. The result of the join is grouped based on the combination of the candidate table, its join key, and numerical column (Line 6). Then, we sort each triplet by their QCR score (Line 7) and return the top-k tables (Line 8). As $n_{II}+n_{IV} = N-(n_I+n_{III})$, we calculate QCR as $\frac{2\times(n_I+n_{III})-N}{N}$, which in SQL is implemented as:

```
(2× SUM(((keys.C_Values IN ($k_0$) AND nums.Quadrant
    = 0) OR (keys.tokenized IN ($k_1$) AND nums.
    Quadrant = 1))::int)- COUNT(*)) / Count(*)
```

k_0 are the input join keys whose corresponding target value is below the average and *k_1* represent join keys whose corresponding target values are greater or equal to the average of the target column. The splitting happens before invoking the query while parsing the input table.

Our implementation has three advantages over the original implementation [49]: **i)** We calculate the absolute value for QCR in one run, without the need to calculate positive and negative correlations twice. **ii)** Our implementation supports numerical join keys as opposed to the baseline. **iii)** Our index does not require quadratic complexity to store the QCR index.

VII. Defining and Optimizing Discovery Plans

By defining seekers and connecting them with combiners, users can declaratively define novel discovery plans. Hence, there will be potentials for optimization through reordering of the operators. To this end, we propose an optimization approach to execute each plan efficiently.

A. Composing discovery tasks with BLEND

We now explain how to assemble a task using BLEND's API. BLEND covers a wide range of discovery tasks:

Simple tasks can contain a single seeker. A user can perform single-column join [69], Multi-column join [24], and Correlation search [49], [48], [23] tasks, simply by using individual seekers dedicated to each task.

Union search aims at discovering tables that can be unioned with the query table, requires a combination of seekers and combiners. The Union discovery plan in BLEND comprises multiple SC seekers, one per input table column, and one *Counter* combiner that aggregates the results of these seekers. We choose a higher top-k limit for individual seekers than the combiner to include tables become relevant when multiple columns are considered in combination [45].

With the operators described above, BLEND is already more expressive than any state-of-the-art system. Furthermore, it allows for any combination of these operators with combiners.

Multi-objective data discovery is the task of finding tables from the lake that can enrich the query table with additional rows and columns as well as filling the empty cells.

```
def build_search_plan(keywords, examples, queries):
  plan = Plan()
  # Keyword Search
  plan.add('kw', Seekers.KW(keywords, k=10))
  # Union Search
  for clm in examples.columns:
    plan.add(clm, Seekers.SC(examples[clm], k=100))
  plan.add('counter', Combiners.Counter(k=10),
    examples.columns)
  # Data Imputation
  plan.add('examples', Seekers.MC(examples, k=10))
  plan.add('query', Seekers.SC(queries, k=10))
  plan.add('intersection', Combiners.Intersect(k=10)
    , ['examples', 'query'])
  # Correlation Search
  plan.add('correlation', Seekers.Correlation(
    examples.joinkey, examples.target, k=10))
  # Results Aggregation
  plan.add('union', Combiners.Union(k=40), ['kw', '
    counter', 'intersection', 'correleation'])
  return plan
```

Listing 4: Building search plan in BLEND.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:53 UTC from IEEE Xplore. Restrictions apply.

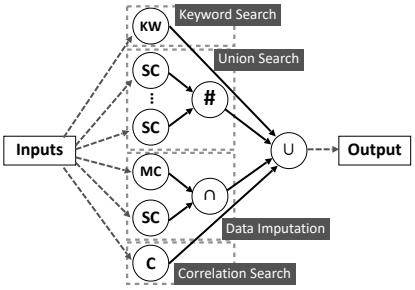


Fig. 4: Multi-objective discovery plan.

Listing 4 shows the plan definition for this task using BLEND's API. The task is comprised of keyword search, union search, data imputation, and correlation discovery.

We initialize the plan object in Line 2. In Line 4 we create the keyword seeker and add it to the plan. Next, we add the union search sub-plan. We add one SC seeker per input column (Lines 6 - 7), followed by the *Counter* combiner (Line 8).

Next, we add a data imputation sub-plan. To find tables with complete rows, we use an MC seeker (Line 10) and for rows where only values of a single column exist, we use an SC seeker (Line 11). We combine the results of both seeker types in Line 12 with an intersection combiner. Similarly, we create the correlation seeker (Line 14). Finally, we aggregate the results of each sub-plans via a *Union* combiner (Line 16). Fig. 4 illustrates the DAG representation of this plan. We introduce more complex tasks in Section VIII.

B. Plan Optimization and Execution

We describe how BLEND optimizes and executes a user-defined discovery plan to solve **Pr4** as discussed in Section II.

1) Naive Optimization: To run a plan, the system needs to execute a sequence of SQL queries generated for each seeker and use combiners to aggregate their results. There are two naive solutions to streamline the queries:

- Executing concurrent seekers independently and aggregating the results from each SQL statement or
- Pushing down the combiners to the database.

The first approach does not use inter-query dependencies to speed up execution. As depicted for the data imputation subplan in Fig. 4, the results of seekers could be intersected, which provides the option of reusing the results of one seeker to limit the search space of a concurrent seeker. The second approach also cannot benefit from such optimization potentials, as SQL *set* operators inside the database are oblivious to the size of subqueries that are its operands. They optimize each subquery in isolation. Moreover, some seekers, such as the MC seekers, can include additional application-level code. To address these problems, BLEND employs query rewriting.

2) The Optimizer in BLEND*:* The optimizer in BLEND receives a discovery plan as input and returns a high-level execution plan as output. This execution plan contains instructions regarding the ranked sequence of SQL statements, how intermediate results can expedite the next query for faster in-DB execution, and how to apply query re-writings to assist the lower-level DBMS optimizer.

The heuristic of the optimizer is that executing the faster seeker first and using the obtained intermediate results to limit the search space for the next seeker increases the efficiency. According to our experiments, this heuristic is effective in 96% of the 4000 randomly sampled cases. The optimizer undergoes four steps: Execution Group (EG) identification, EG ordering, operator ranking, and query rewriting.

EG identification. To ensure that the optimizer preserves the semantic of the original plan, we discover isomorphic plans that deliver the same output. To this end, the optimizer identifies seekers that can be re-ordered without changing the plan's output. Note that although seekers that are connected to the same combiner can be reordered, we only change the order of seekers connected to the *Intersection* combiner. The remaining combiners are either non-commutative, i.e., *Difference*, or they cannot benefit from the operator re-ordering, i.e., *Union* and *Count*. The optimizer traverses the DAG of the discovery plan and identifies all of such so-called execution groups (EG) and builds a hyper-DAG on top of the original DAG.

EG ordering. Once the EGs are discovered, the optimizer follows a topological order based on the dependency of the EGs and processes one EG after another. An EG is selected only after all its dependencies are executed. Once the next EG is selected for execution, BLEND optimizes the sub-plan of the EG before executing its seekers in the database.

Operator ranking. Following the heuristic of executing faster seekers first, BLEND ranks the seekers based on their expected runtime, which depends on three crucial parameters:

1) the **seeker type**, as each type has a different runtime complexity which can be determined through apriori analysis,
2) the **number of rows projected by** Q, and
3) the **frequency of values from** Q **in the database**.

Using these parameters, we build a two-step re-ordering approach: rule-based ranking and learning-based cost estimation. Similar to logical optimizers in databases, we first reorder seekers based on types. According to the implementation details of the operators in Section VI, the KW operator has the lowest complexity among operators. It requires a single index scan, during which *Cell Value*s are compared to Q. If there are n elements in the index and $|Q|$ is the number of keywords Q_{KW}, KW complexity is $O(n \cdot |Q|)$. Although, SC yields the same complexity of $O(n \cdot |Q|)$, the input size $|Q_{\text{SC}}|$ for SC is a column compared to a few keywords in KW, making $|Q_{\text{SC}}| \geq |Q_{\text{KW}}|$. The correlation operator (C) requires two index scans, one to find overlapping columns with join key leading to the same complexity as SC ($O(n \cdot |Q|)$), and one scan to find candidate numerical columns $O(n)$. The operator joins the results of these scans, which is linear for hashjoins, resulting in overall complexity of $O(3{\cdot}n{\cdot}|Q|)$. Finally, the MC operator requires x index scans, one for each input column, and $x-1$ hashjoins to combine the results, therefore, it has the highest complexity of $O(x \cdot n \cdot |Q| + (x-1) \cdot 2n)$, where x is the number columns in the input composite key and $x \geq 2$. Furthermore, MC has additional application-level code,

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:53 UTC from IEEE Xplore. Restrictions apply.

adding additional overhead. Hence, we can summarize the execution order with the following rules:

- **Rule 1.** The keyword operator always executes first.
- **Rule 2.** The MC seeker always executes last.
- **Rule 3.** SC is prioritized over C.

We implement the rules as `if-else` clauses in the optimizer.

Learning-based cost estimation. For seekers of the same type, the runtime depends on Q and the frequency of the input values in the database. We train a regression model per seeker type to estimate the relative runtime. We encode the input using three features: *cardinality of* Q, *the number of columns involved in* Q, and *the average frequency of values from* Q *in the database*. For MC, the average frequency feature is calculated by multiplying the average frequencies of columns in Q. This is because the SQL query for MC performs a join between the obtained index elements for each column.

For training each seeker type regression model, we randomly sample 1000 input Qs from the Gittables data lake (See Table II). Then, we execute the seekers independently and measure the execution runtime to collect the ground truth. Training occurs offline during deployment, but predicting and ordering seekers is part of the online optimization process. The training time on the largest lake was 60 seconds. Generally, one could assume that the training is transferable but it is advisable to run the training module once upon lake installation.

Query rewriting. Once the seekers are ranked, BLEND starts with the fastest seeker and uses the intermediate results of each executed seeker to speed up the next one. BLEND rewrites the SQL queries by adding combiner-dependent predicates that check the intermediate results of the previous step:

- *Intersection*: `WHERE TableId IN (...)`
- *Difference*: `WHERE TableId NOT IN (...)`
- *Count*: `GROUP BY TableId ORDER BY COUNT(*)`
- *Union*: No rewriting

For the *Count* combiner a `GROUP BY` is needed to count the occurrence of tables and descendingly order the results by their frequencies. For *Union* combiners a rewriting is not needed. This re-writings are implemented via replacing predefined placeholders in each seekers' default SQL statement.

Example 2. Consider the data imputation sub-plan in Fig. 4. First, the optimizer identifies independent seekers. Both the *MC* and the *SC* seekers are connected to the same *Intersection* combiner, thus, they form an EG. As *SC* is faster (**Rule 2**), BLEND prioritizes its execution. The *MC* seeker is rewritten using the table identifiers obtained by the *SC* seeker as shown in Listing 2 via a limiting predicate:

```
WHERE Q1_index_hits.TableId IN (IR_SC).
```

IR_{SC} is the list of table identifiers discovered by the *SC* seeker. After query rewriting, the operator is executed. •

In-DB Optimization BLEND 's optimizer acts before the in-DB optimizers, it complements in-DB optimizers by revising discovery plans and rewriting queries before execution.

Theorem 1. *Given the proposed combiners and seekers,* BLEND*'s optimizer does not alter the output of the query.*

TABLE II: Data lakes used in the experiments.

Data lake	*Tables*	*Columns*	*Rows*
DWTC [2]	145M	760M	1.5B
Lakebench Webtable Large [18]	2.8M	14.8M	63.7M
Gittables [33]	1.5M	16.8M	345M
German Open Data [1]	17,144	440K	62M
WDC [4]	-	163M	1.6B
Canada, US, and UK Open Data [69]	-	745K	1.1B
TUS [45]	1,530	14.8K	6.8M
TUS Large [45]	5,043	55K	9.6M
SANTOS [34]	550	6,322	3.8M
SANTOS Large [34]	11,090	121K	85M
NYC open data [3]	1,063	16K	290M

Proof. The query is subject to operator ranking and rewriting. Operator ranking only applies to seekers connected by an *Intersection* combiner, which is commutative, so the order of execution is irrelevant. The rewriting rule for *Intersection* ensures that only results from intersected seekers are included, preserving correctness. For *Difference*, the re-writing rule filters out the previously discovered tables, aligning with the difference operation. The rule used for the *Count* combiner only counts the frequency of table identifiers and does not alter the underlying data. *Union* requires no rewriting. Thus, the optimizer preserves the correctness of queries. □

VIII. EXPERIMENTS

We carried out a series of experiments to compare BLEND's efficiency and effectiveness for executing complex as well as simple discovery tasks with individual stand-alone techniques and their ad-hoc compositions. We also conducted a user study on the needs and preferences of data practitioners.

A. Experimental Setup

We utilized ten data lakes to compare each higher-level operator to its baseline on the benchmarks from corresponding original papers. Table II provides details about each of these lakes. WDC [4] and Canada-US-UK Open Data [69] are used to benchmark Josie and only contain column identifiers, therefore, the number of tables is not known.

1) BLEND *Implementation and Deployment:* We evaluate BLEND on top of two different database engines, PostgreSQL (row store) and a commercial column store. If not explicitly mentioned, both BLEND and the baselines run on the commercial column store. Also, the default value for k is 10. We run all experiments on a machine with 64 processing cores, 512GB of main memory, and 10TB SSD storage. The code for BLEND is available on our GitHub repository[1].

2) Operators And Baselines: Since we propose a holistic system, we also focus on holistic evaluation. Yet, we also evaluate BLEND on well-known tasks: Single- and multi-column join search, union search, correlation search, and a set of more complex tasks. As there is not a single discovery system to cover all of the discovery operators, we compare to individual state-of-the-art baselines per task.

Single-column join search. We use Josie [69] as the state-of-the-art single-column join discovery baseline. Note that we do

[1]https://github.com/LUH-DBS/Blend

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:53 UTC from IEEE Xplore. Restrictions apply.

not evaluate keyword search because it is an attribute-agnostic version of the join search. **Multi-column join search.** We use MATE [24], the only approach to discover multi-column joins, as the baseline. **Union Search.** We select Starmie [25], the most recent and the most effective approach, as the baseline for our union discovery plan. **Correlation search.** We use the sketch-based correlation approximation approach [49] as the baseline for our correlation discovery plan. This baseline is the most efficient correlation search approach.

B. Complex Discovery Tasks

We first evaluate the capabilities of BLEND in facilitating the creation of several complex pipelines, namely, *data discovery with negative examples*, *data imputation*, *multicollinearity-aware feature discovery*, and *multi-objective data discovery*. While these pipelines may not cover every possible discovery plan that one can implement using BLEND, they represent commonly discussed discovery applications in the state-of-the-art [43], [5], [64], [68], [67].

We implement each task once with BLEND and once with a federation of available state-of-the-art systems. We further compare the runtime with and without optimization in BLEND. ***B-No*** represents BLEND without optimization. The results are shown in Table III.

1) Metrics: We compare the runtime, lines of code (LOC), the number of systems involved in the implementation, and the required number of index structures.

2) Data Discovery with Negative Examples: In this task, we want to discover tables with disjoint information by specifying negative examples that must be absent in the discovered tables.
Baseline implementation. Here, we use MATE and application code. MATE filters irrelevant tables based on a set of examples. The remaining tables are then validated in application code row-by-row to drop tables with negative examples.
BLEND implementation. We use two MC seekers and a *Difference* combiner. The first seeker discovers tables based on the query table and the second seeker finds tables comprising the negative examples, which are then filtered by the combiner.
Results. We use the SANTOS benchmark with 365 queries. We leverage the ground truth unionable tables that do not completely overlap with the query tables and use a random set of non-overlapping rows as negative examples resulting in $1k$ negative examples on average. BLEND is on average twice as fast as the baseline. The row-by-row validation phase is the main bottleneck of the baseline and B-No. BLEND rewrites the SQL queries using tables containing negative examples, filtering the irrelevant tables in the database and benefiting from the DBMS-level optimization. Although the baseline has only one data discovery system, the user needs to develop a validation phase on top of MATE. This leads to over 14 times more LOC for baseline with 72 LOC compared to only 5 LOC required to define the plan in BLEND.

3) Example-Based Data Imputation: A common strategy to find missing values is to utilize functional dependencies [5]. This requires discovering tables that contain the non-missing row values as well as rows that contain the incomplete rows.
Baseline implementation. As an ad-hoc solution, one can leverage MATE to first identify tables that contain complete rows, and Josie to discover partial rows. The intersection of the fetched tables is used to infer the missing values.
BLEND implementation. Using BLEND, we simulate the same process with the data imputation sub-plan in Fig. 4.
Results. We select over $1,000$ random column pairs from Gittables, consider the first 5 rows as the complete examples, and delete the values of the second column for the remaining rows. As shown in Table III, BLEND is over $3.5X$ faster than the baseline. The optimization step re-writes the MC operator to limit the search space to only tables that contain the query values. The two approaches also differ in terms of LOC. BLEND only requires 5 lines of code compared to 51 that are necessary to align the libraries of MATE and Josie, each written in different language. Also, the baseline requires two sets of incompatible indexes to enable data imputation.

4) Multicollinearity-Aware Feature Discovery: This is the task of obtaining new features to enrich a dataset for a prediction task. The new features should correlate with the input target but not with each other. The QCR approach [49] discovers correlating features with a single column. To avoid multicollinearity of the features we need to further expand this.
Baseline Implementation. To avoid multicollinearity of features, we apply multiple rounds of the QCR approach, once to discover correlating features with the target and then for every feature in the input dataset. Each time we filter the tables that were results of the previous iteration. In addition, we use MATE to ensure joinability with the join column sets. The final output is the intersection of these two sets of tables.
BLEND Implementation. Using BLEND, one can accordingly use multiple correlation seekers and one MC seeker. The filtering between the correlation seekers is enacted by using a *Difference* combiner after each multicollinearity check.
Results. We chose 6 datasets from the SANTOS lake with two joinable string columns and two numerical columns each, one feature and one target. As shown in Table III, BLEND is $2.5X$ faster than the baseline and $1.8X$ faster than B-NO. The baseline also needs 49 lines of application code for alignment and filtering, compared to just 7 lines with BLEND.

5) Multi-Objective Data Discovery: Our last use case is a subset of the complex plan described in Section VII-A. We exclude data imputation, as we have evaluated it before.
Baseline Implementation. This pipeline requires combination of three systems Josie [69], Starmie [25], and QCR [49]. This needs building three sets of indexes, each requiring a different process from building massive posting lists to training a contrastive model. Then, one has to compile these approaches, each written in different language, Josie in `GO`, Starmie in `Python`, and QCR baseline in `JAVA`. Starmie vectors are stored as a file, QCR performs the best if stored in a column store and Josie is tightly coupled with PostgreSQL.
BLEND Implementation. The implementation of the multi-objective discovery is shown in Listing 4. Note that we exclude data imputation (Lines 10-12).

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:53 UTC from IEEE Xplore. Restrictions apply.

TABLE III: Experimental results on complex discovery tasks. B-NO does not use the plan optimizer.

	With Negative Examples			*Data Imputation*			*Feature Discovery*			*Multi-Objective Discovery*		
	BLEND	**B-NO**	**Baseline**	**BLEND**	**B-NO**	**Baseline**	**BLEND**	**B-NO**	**Baseline**	**BLEND**	**B-NO**	**Baseline**
Runtime	**14.2**	111.9	30.5	**0.19**	0.26	0.67	**9.0**	16.3	22.8	**5.5**	**5.5**	47.0
LOC	5	5	72	5	5	51	7	7	49	8	8	135
# of Systems	1	1	1	1	1	2	1	1	2	1	1	3
# of Indexes	Single	Single	Multi	Single	Single	Multi	Single	Single	Multi	Single	Single	Multi

TABLE IV: Experiment on optimizer effectiveness.

	Runtime (Seconds)			*Runtime Gain*		*Accuracy*	
Seeker	**Rand**	**BLEND**	**Ideal**	**BLEND**	**Ideal**	**BLEND**	**Ideal**
Mixed	5.1	2.0	1.2	61.1%	75.9%	84.4%	100%
SC	1.3	1.0	0.9	21.5%	26.9%	99.8%	100%
MC	17.2	4.5	3.1	73.7%	82.2%	70.3%	100%
C	5.2	1.3	1.1	74.8%	77.8%	89.5%	100%

Results. BLEND requires fewer lines of code and a single index to implement this complex discovery plan, demonstrating its ability to facilitate the development of complex and on-demand pipelines without either data- or code-level complexities. BLEND and the baseline yield 5.5 and 47 seconds runtime, respectively. BLEND is more efficient because it pushes the operators down to the database and prevents time-consuming data loading between the database and memory. As expected, the runtime for BLEND and B-NO are equal for this plan, because the sub-plans are connected via a *Union* combiner.

C. The Optimizer

We measure the runtime improvement and the accuracy achieved through rule- and ML-based optimization.

1) Metrics: We compare the execution time for three approaches: without the optimizer (executing seekers in a random order), with the optimizer, and with an oracle optimizer that consistently identifies the faster seeker. The reported runtimes for BLEND also include the optimization overhead. Finally, the *Accuracy* shows the percentage of cases in which the optimizer correctly selects the faster seeker. The ground truth is obtained through an exhaustive search by executing all combination of operator ranking and labeling the fastest order of seekers as the positive case and the remaining cases as negatives.

The results are summarized in Table IV. Each experiment involves 100 random plans of two seekers connected with an *Intersection* combiner. Gittables serves as the target lake as well as the source of random inputs for the seekers. *Seeker* represents the type of seekers within the plans.

2) Rule-Based Optimizer: The first row in Table IV shows the effect of the rule-based optimizer, because the seeker types are different. The results show that the rules optimally re-order the seekers in 84.4% of the cases. This leads to a runtime gain of 61.1% over the baseline with random order.

3) ML Optimizer: The last three rows in Table IV highlight the efficacy of the ML optimizer. On average, BLEND achieves a runtime improvement of 21.5%, 73.7%, and 74.8% for SC, MC, and correlation seekers, respectively. The gap between the ideal runtime and BLEND's runtime for the MC seeker is bigger because the runtime for MC also depends on the number of filtered candidates and the application-level code.

4) Assessing the Reliability of the Optimizer: Similar to database optimizers, our optimizer is not 100% accurate and can lead to an incorrect order of operators [46]. To assess the reliability of our optimizer, we conducted a statistical hypothesis test on its observed accuracy and runtime improvement.

The null hypothesis (H_0) is that our optimizer's superior accuracy is due to chance, while the alternative hypothesis (H_1) suggests that the optimizer is significantly better than random. Using a z-test, we compare our optimizer's average success rate of 86% ($\hat{p}$) across 4000 (n) queries against the random success rate of 50% under the null hypothesis ((p_0)). Z-test is calculated as: $\frac{\hat{p}-p_0}{\sqrt{\frac{p_0(1-p_0)}{n}}} \approx 45.6$. Obtaining the z-value, we can calculate: p-value $= 2 \times (1 - \Phi(|z|)) \approx 0$, where Φ is the cumulative distribution function of the normal distribution. This let us to reject the null hypothesis, concluding that our optimizer's superiority is statistically significant.

D. Single-Column Join Search

We compare our join discovery operator to Josie. We run the experiments on the WDC, Canada-US-UK Open Data, and the Gittables data lake. We follow the same approach as the reference paper [69] to generate query workloads, resulting in $3{,}000$ query columns per data lake, $1{,}000$ per query size.

1) Metrics: We evaluate both approaches based on runtime, precision@k and recall@k. WDC and open data lakes are not labeled, therefore, they are only used for the runtime experiments. We use LakeBench benchmark [18] to measure the effectiveness of the approaches.

2) Results: Fig. 5 illustrates the results. To indicate the storage format, we append *(Column)* (for column-store) and *(Row)* (for PostgreSQL) to the approach names. The x-axis shows the maximum query size of each query batch, which varies due to different table sizes in the lakes. Fig. 5 shows that the column-store version of BLEND consistently outperforms Josie. This is because Josie lacks some optimizations that are tightly coupled with specific data types on PostgreSQL. In the PostgreSQL experiments, except for very large (over $100k$) queries on the Canada-US-UK lake, Josie outperforms BLEND. Still, the runtime difference is on average below a second.

We also evaluated our approach on the LakeBench benchmark [18]. Fig. 6.a depicts the runtime comparison between BLEND, Josie, and DeepJoin [21]. The results show that DeepJoin achieves the best runtime thanks to its efficient HNSW index. BLEND and Josie achieve the same effectiveness results as their outputs are identical. As DeepJoin is a semantic join discovery approach, it achieves higher effectiveness compared to BLEND and Josie that are designed to discover equi-joins.

E. Multi-Column Join Discovery

We compare the MC operator in BLEND to MATE. We use the same benchmark as in the original paper [24], namely

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:53 UTC from IEEE Xplore. Restrictions apply.

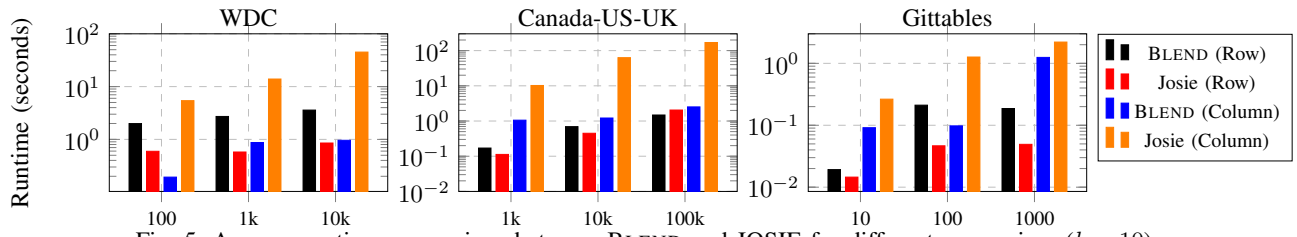
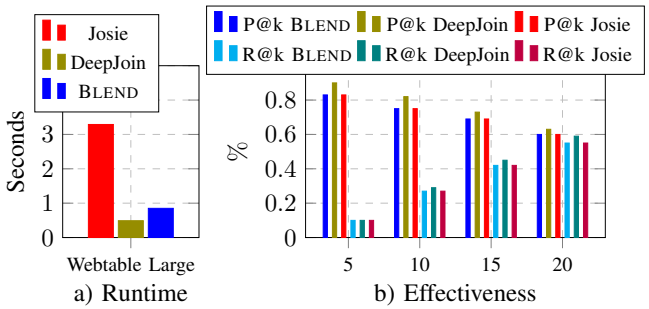


Fig. 5: Average runtime comparison between BLEND and JOSIE for different query sizes (k = 10).



a) Runtime b) Effectiveness

Fig. 6: Lakebench experiments.

TABLE V: Precision comparison of BLEND and MATE.

Lake	BLEND			*MATE*		
	TP	**FP**	**Precision**	**TP**	**FP**	**Precision**
Open Data	5,563	**14**	**99.7%**	5,563	3,587	61%
DWTC	21,228	**3**	**99.99%**	21,228	7,953	73%

DWTC and German open data. The benchmark contains 450 query tables randomly selected from each lake.

1) Metrics: We compare both approaches based on their runtime, the average number of true positives (TP), false positives (FP), and precision. A TP is a row that is joinable to an input row on the defined composite key. A FP is a row that does not fully overlap with the join key on all columns and therefore is not joinable with any input row. To find the ground truth, we validate every candidate row by comparing the exact values to the corresponding row in the query table.

2) Results: BLEND is over $2.6X$ and $10X$ faster than MATE on DWTC and German open data, respectively. It is faster because the SQL statement in the MC seeker has a higher filtering effect than the baseline. To understand this runtime gap, Table V shows the average TP, FP, and precision for both approaches. Recall for both approaches is 100% due to bloom filter character. BLEND achieves over 99% precision on both lakes. In particular, the average number of obtained rows is 23,326 and 2,237,671 for BLEND and MATE respectively for DWTC, and 5,746 and 215,780 for open data.

F. Union Search

We compare the union discovery plan of BLEND to Starmie. The experiment involves four data lakes from the Starmie paper [25]: SANTOS, SANTOS Large, TUS, and TUS Large. They contain 50, 80, 150, and 100 query tables, respectively.

1) Metrics: In addition to the runtime of the approaches, similar to the previous works [25], [34], [45], we use precision@k, recall, and Mean Average Precision@k (MAP) to measure the quality of the results. For the quality metrics,

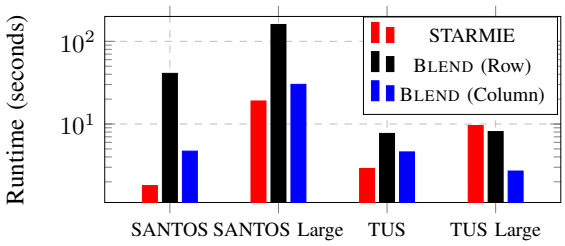


Fig. 7: Time comparison between BLEND and STARMIE.

for each k and data lake, the **bold** value shows the approach with a higher score. We use the ground truth introduced in the original papers [25], [45], [34].

2) Results: Fig. 7 shows that Starmie outperforms BLEND in all cases except TUS Large due to its fast contrastive model and in-memory HNSW index. BLEND (Column) is one order of magnitude faster than BLEND (Row). As Starmie considers both syntactic similarities and semantic associations, the final results of the approaches are different. Therefore, we also compare the quality of the results. Note that SANTOS Large does not contain ground truth labels. According to Table VI, Starmie slightly outperforms BLEND for $k = 10$. This is because some unionable tables with high semantic relatedness significantly lack overlap similarity. For $k = 20$, BLEND and Starmie perform similarly. For higher k values BLEND achieves better results in all cases. Both approaches display low recall for small k values on the TUS and TUS Large lakes. This is because the ground truth for these benchmarks has a large number of unionable tables. For $k = 10$, the ideal recall is 6% and 4% for TUS and TUS Large respectively. Overall, the results show that BLEND's native union operator is competitive with state-of-the-art.

G. Correlation Discovery

We compare the correlation operator of BLEND with the QCR baseline [49] on the NYC lake. As our approach avoids the mechanics for random sampling during runtime and follows a convenience sampling approach, we also test a version where the indexed rows are apriori shuffled. We call it BLEND (rand). The first benchmark from the QCR paper, i.e., NYC (Cat.), comprises queries with one categorical join key and one numerical target column. For the second benchmark, i.e., NYC (All), we allow for the join key to be selected from any column type, not restricted to only categorical columns. This is because the join keys are not restricted to categorical columns, e.g., all the keys in the TPC-H benchmark are numerical.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:53 UTC from IEEE Xplore. Restrictions apply.

TABLE VI: Quality comparison between BLEND and STARMIE.

Lake	*k=10*						*k=20*					
	BLEND			*STARMIE*			BLEND			*STARMIE*		
	P@k	**Recall**	**MAP**	**P@k**	**Recall**	**MAP**	**P@k**	**Recall**	**MAP**	**P@k**	**Recall**	**MAP**
SANTOS	92%	70%	94%	**97%**	**73%**	**99%**	**100%**	89%	94%	97%	**95%**	**98%**
TUS	94%	05%	94%	**95%**	**06%**	**98%**	**95%**	10%	94%	92%	10%	**96%**
TUS Large	**94%**	04%	94%	93%	04%	**97%**	**94%**	07%	94%	90%	07%	94%
	k=50						*k=100*					
TUS	**96%**	**24%**	**95%**	90%	23%	93%	**93%**	**43%**	**92%**	81%	38%	90%
TUS Large	**92%**	**17%**	**93%**	85%	15%	90%	**92%**	**32%**	**92%**	77%	26%	85%

TABLE VII: Experiment on Correlation-based discovery.

	BLEND		BLEND *(rand)*		*Baseline [49]*	
Lake	**P@10**	**R@10**	**P@10**	**R@10**	**P@10**	**R@10**
NYC (All)	42%	40%	**44%**	**42%**	24%	23%
NYC (Cat.)	55%	57%	60%	58%	**64%**	**61%**

1) Metrics: We measure the runtime, effectiveness (precision@k and recall@k), and ablate the sampling method. To find the ground truth, we calculate the exact correlation between the query target and each candidate from the data lake and identify the top-k correlating columns.

2) Results: Table VII shows the P@k and recall for BLEND, BLEND (rand), and the baseline for $k = 10$ and $h = 256$. On the NYC (All) benchmark, BLEND outperforms the baseline by 18% and 17% for P@k and R@k, respectively. This performance gap is because the baseline cannot discover the correlating columns when the join key is numeric. On NYC (Cat.), the baseline achieves slightly higher P@k and recall. This is due to their differences in hashing and sketching. Further, random sampling (BLEND (random)) outperforms the vanilla BLEND, which uses convenience sampling based on `LIMIT h` sorted by RowId. As data sorted by RowId follows an unknown order, consecutive values might be duplicates making this sampling non-representative. On NYC (Cat.) the average runtime in seconds is 0.67 and 0.86 and on NYC (All) 0.56 and 1.33 for the baseline and BLEND, respectively. In summary, the correlation seeker of BLEND is competitive while being flexible with regard to the type of the join column and also allowing sketch sizes to be specified on-demand.

H. Index Size in BLEND

Table VIII compares the storage required for BLEND's index and the combination of DataXFomer, Josie, MATE, Starmie, and QCR indexes. The index for WDC and Canada-US-UK lakes excludes the QCR index as they lack table identifiers. On average, BLEND requires 57% less storage compared to the combination of the state-of-the-art indexes. The index construction time of BLEND ranged between 2 to 80 hours for Gittables and DWTC, respectively.

I. User Study

To assess the usefulness of our system and the validity of our premises we conducted a survey and obtained results from 18 expert participants (40% response rate) covering sectors, such as healthcare, banking, technology, and education. The detailed results can be found on our GitHub repository[2].

[2]https://github.com/LUH-DBS/Blend/blob/main/UserStudy.md

TABLE VIII: The storage required for BLEND and S.O.T.A.

Data lake	BLEND	*Combination of S.O.T.A indexes*
DWTC	**998.0 GB**	3.6 TB
Gittables	**397.3 GB**	511.0 GB
German Open Data	**79.7 GB**	575.7 GB
WDC	**151.8 GB**	223.7 GB (W/O QCR)
Canada, US, and UK	**114.7 GB**	230.1 GB (W/O QCR)
TUS	**7.9 GB**	8.4 GB
TUS Large	**12.2 GB**	13.1 GB
SANTOS	**5.5 GB**	6.0 GB
SANTOS Large	**135.0 GB**	144.2 GB
NYC open data	**709.2 GB**	763.0 GB

TABLE IX: Statistics obtained from the conducted user study.

	Research	**Industry**	**All**
Number of **participants**	9	9	18
Question 1. How often do you find data within a single search?			
Rarely (0%) - Often (100%)	27.5%	38.8%	33.3%
Question 2. Is a single discovered table sufficient as the output of the discovery task?			
Yes | No	11% | **89%**	00% | **100%**	06% | **74%**
Question 3. What are your most frequent data discovery tasks?			
Discovery for rows	33%	**67%**	**50%**
Correlation discovery	**44%**	56%	**50%**
Join discovery	**44%**	33%	39%
Keyword search	**44%**	33%	39%
multi-column join discovery	33%	22%	28%
Question 4. How do you solve data discovery tasks?			
With custom scripts	**100%**	**56%**	**78%**
Writing SQL queries	44%	**56%**	50%
Asking people	33%	**56%**	44%
Using open source tools	56%	33%	44%
Using commercial tools	22%	22%	22%
Question 5. What programming language do you prefer?			
Python	**100%**	**89%**	**94%**
Java | SQL | C++	78% | 78% | 56%	**89%** | 78% | 78%	83% | 78% | 67%
Question 6. Where do you store your data lake?			
DBMS | File systems | Both	33% | **44%** | 22%	44% | 00% | **56%**	**39%** | 22% |**39%**
Question 7. Would you use DBMS if indexing and optimizations are provided?			
YES | NO	**100%** | 0%	**100%** | 0%	**100%** | 0%
Question 8. Which API do you prefer for simple tasks?			
BLEND | Python | SQL	34% | 22% | **44%**	**56%** | 11% | 34%	**44%** | 17% | 39%
Question 9. Which API do you prefer for complex tasks?			
BLEND | Python	**89%** | 11%	**89%** | 11%	**89%** | 11%

A concise representation of the results of the user responses is depicted in Table IX. The survey reveals that our expert participants rarely find desired datasets within a single search (question 1). Rather, their discovery needs are often met by composition of tables (question 2). The two most common tasks were discovering tables *containing a set of rows* and *containing a correlating column to a target*. These tasks are more prevalent in industry (question 3). 78% of the participants solve such problems via custom scripts from scratch, with 94.4% using Python (questions 4 and 5). Moreover, 78% indicated that their data lakes are managed at least partly, via DBMS; 39% reported to only use DBMS (question 6). Industrial participants showed a higher preference for storing the data lake in a DBMS compared to academics. This is because research often prioritizes expeditious results, while in industry the emphasis is on stable and maintainable solutions. All respondents unanimously expressed that they would use a DBMS if data discovery capabilities such as, indexes and optimizations are provided (question 7). The answers to the

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:53 UTC from IEEE Xplore. Restrictions apply.

previous questions underpin the motivation for our approach.

We also surveyed users on their preference for simple and complex task implementations. The simple task involved querying tables with specific keywords, while the complex task aimed at finding joinable tables on two columns that also have correlating columns to a target. For both tasks, we provided implementations via BLEND and alternatives. The simple task was presented in BLEND's API, Python, and SQL, while the complex task could only be implemented using BLEND's API and Python. For the simple task, the majority (44.4%) favor BLEND followed by SQL (38.9%). For the complex task, this gap is larger (89% vs. 11% in favor of BLEND), as the Python implementation becomes significantly complex. Participants justify their choice through BLEND's more flexible, concise, easy-to-understand, and robust API (questions 8 and 9).

IX. Related Work

Our system BLEND relates to several lines of research.

Data discovery in data lakes. Several large companies have reported about internal platforms and engines to retrieve structured information from their in-house data lakes. LinkedIn published WhereHows [38], Microsoft introduced DLN [10], the Apache foundation offers Atlas [7], and Google presented Goods [30] and Google Dataset Search [12]. These systems record the life-cycle of datasets through metadata and lineage, where discovery operators are generally underexplored.

Fernandez et al. [26] presented Aurum, a data discovery system utilizing an enterprise knowledge graph to connect data lake tables based on joinability. The system is capable of answering keyword queries. It also allows for further exploration of the joinable tables based on pre-calculated join similarities in the data lake. Moreover, Aurum provides a domain-specific language to facilitate browsing of related tables. Because of the precaluclation, Aurum lacks the capability for "table as query" search. BLEND provides more low-level operators beyond single-column join discovery and keyword search that can be declaratively composed to a plan and optimized.

Individual Discovery Techniques. In addition to the aforementioned approaches, several studies introduce and utilize index structures for discovery tasks, e.g., union search [8], [45], [11], [25], [51], [68], join search [68], [51], [69], [63], [62], [59], [27], [70], [16], [56], [24], [50], [26], [13], [8], keyword search [12], [66], [13], [26], correlation search [23], [9], [49], [48], [17], and augmentation by example [5], [64], [35], [6], [28]. Among these techniques, some are closely related to our techniques. Starmie [25] discovers unionable tables using an ensemble of indexes. It leverages a contrastive model and a fast HNSW index to find the most similar column embeddings. Josie [69] leverages two indexes for efficient join discovery using a set of data-dependent pruning techniques. COCOA [23] uses a combination of inverted index and *Order index* to find non-linear correlating columns from data lake in linear time. MATE [24] is the only approach that addresses the problem of multi-column join discovery. It introduces a hash-based inverted index and a filtering strategy that facilitates discovery of relevant tables based on a composite join key. Each of these approaches address a single task. In BLEND, we combine indexes of the prevalent tasks.

Indexes for semantic data discovery. To facilitate semantic data discovery [20], [16], [56], [10], [65], [37], [60], [25], indexes for high dimensional embeddings are proposed. HNSW [39], [25] and IVF-PQ [47] are indexes to accelerate vector search in finding similar elements embedded as high-dimensional vectors. Although these structures enable efficient nearest neighbor search on vectors, they are not easy to integrate within our holistic token-based index. Further, their approximate nature make their inclusion into our optimizer non-trivial as reordering might change the result set.

Optimizing concurrent queries. Multi-query optimizations (MQO)s [57], [52], [32], [53] aim to create a single global query to avoid duplicate sub-queries. Our seekers contain complex predicates compared to simple point queries, making global query creation impractical due to rare overlaps in queries. Our optimizer uses query rewrites that uses intermediate results to limit the search space for subsequent queries, regardless of overlaps in their logical plans.

DAGs and ETL. Our optimization strategies assume a topological order of operator groups. Existing work on ETL optimization also deals with DAGs [29], [58], [42] that connect input to output schemata through activities [54], [55]. While we borrow ideas from ETL and DB optimization, we address a different optimization goal. The underlying data in our scenario is unknown to the user. The optimization potential is guided by the user query on the data that resides inside the lake. Further our table operations go beyond relational operators that are studied for optimization in ETL pipelines.

X. Conclusion and Future Work

We propose a unified system for customized data discovery tasks that allows for declarative task definition. To this end, we introduce atomic operators to serve as building blocks for higher-level tasks. Furthermore, we present a minimal yet general index that facilitates efficient execution of operators. We also propose a task-level optimization for efficient query execution. Our system does not cover semantic and fuzzy relationships. It would be interesting to extend our system to enable the execution and optimization of these operators. This can include incorporation of high-dimensional embeddings into our index structure.

Acknowledgment

We thank Matthias Böhm for his insightful feedback. This work was supported by the German Research Foundation (DFG) under grant agreement 387872445 and the German Ministry for Education and Research as BIFOLD — "Berlin Institute for the Foundations of Learning and Data" (01IS18025A and ref. 01IS18037A). This work was also partially funded by NSF awards IIS-2107248, IIS-1956096, and IIS-2325632. We acknowledge the support of the Canada Excellence Research Chairs (CERC) program. Nous remercions le Chaires d'excellence en recherche du Canada (CERC) de son soutien.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:53 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] Das datenportal für deutschland. https://www.govdata.de/. Accessed: 2024-07-17.

[2] Dresden web table corpus (dwtc). https://wwwdb.inf.tu-dresden.de/research-projects/dresden-web-table-corpus/. Accessed: 2024-07-17.

[3] Nyc open data. https://opendata.cityofnewyork.us//. Accessed: 2024-07-17.

[4] Wdc web table corpora. https://webdatacommons.org/webtables/. Accessed: 2024-07-17.

[5] Z. Abedjan, J. Morcos, M. N. Gubanov, I. F. Ilyas, M. Stonebraker, P. Papotti, and M. Ouzzani. Dataxformer: Leveraging the web for semantic transformations. In *Proceedings of the Conference on Innovative Data Systems Research (CIDR)*, 2015.

[6] Y. Amsterdamer and M. Cohen. Automated selection of multiple datasets for extension by integration. In G. Demartini, G. Zuccon, J. S. Culpepper, Z. Huang, and H. Tong, editors, *Proceedings of the International Conference on Information and Knowledge Management (CIKM)*, pages 27–36. ACM, 2021.

[7] Apache. ATLAS: Data Governance and Metadata framework for Hadoop. http://atlas.incubator.apache.org, 2016.

[8] R. J. Bayardo, Y. Ma, and R. Srikant. Scaling up all pairs similarity search. In C. L. Williamson, M. E. Zurko, P. F. Patel-Schneider, and P. J. Shenoy, editors, *Proceedings of the International World Wide Web Conference (WWW)*, pages 131–140. ACM, 2007.

[9] J. Becktepe, M. Esmailoghli, M. Koch, and Z. Abedjan. Demonstrating MATE and COCOA for data discovery. In *Proceedings of the International Conference on Management of Data (SIGMOD)*, pages 119–122. ACM, 2023.

[10] S. Bharadwaj, P. Gupta, R. Bhagwan, and S. Guha. Discovering related data at scale. *Proceedings of the VLDB Endowment (PVLDB)*, 14(8):1392–1400, 2021.

[11] A. Bogatu, A. A. A. Fernandes, N. W. Paton, and N. Konstantinou. Dataset discovery in data lakes. In *Proceedings of the International Conference on Data Engineering (ICDE)*, pages 709–720. IEEE, 2020.

[12] D. Brickley, M. Burgess, and N. F. Noy. Google dataset search: Building a search engine for datasets in an open web ecosystem. In L. Liu, R. W. White, A. Mantrach, F. Silvestri, J. J. McAuley, R. Baeza-Yates, and L. Zia, editors, *Proceedings of the International World Wide Web Conference (WWW)*, pages 1365–1375. ACM, 2019.

[13] M. J. Cafarella, A. Y. Halevy, and N. Khoussainova. Data integration for the relational web. *Proceedings of the VLDB Endowment (PVLDB)*, 2(1):1090–1101, 2009.

[14] M. J. Cafarella, A. Y. Halevy, D. Z. Wang, E. Wu, and Y. Zhang. Webtables: exploring the power of tables on the web. *Proc. VLDB Endow.*, 1(1):538–549, 2008.

[15] R. Cappuzzo, G. Varoquaux, A. Coelho, and P. Papotti. Retrieve, merge, predict: Augmenting tables with data lakes. *CoRR*, abs/2402.06282, 2024.

[16] S. Castelo, R. Rampin, A. S. R. Santos, A. Bessa, F. Chirigati, and J. Freire. Auctus: A dataset search engine for data discovery and augmentation. *Proceedings of the VLDB Endowment (PVLDB)*, 14(12):2791–2794, 2021.

[17] N. Chepurko, R. Marcus, E. Zgraggen, R. C. Fernandez, T. Kraska, and D. R. Karger. ARDA: automatic relational data augmentation for machine learning. *Proceedings of the VLDB Endowment (PVLDB)*, 13(9):1373–1387, 2020.

[18] Y. Deng, C. Chai, L. Cao, Q. Yuan, S. Chen, Y. Yu, Z. Sun, J. Wang, J. Li, Z. Cao, K. Jin, C. Zhang, Y. Jiang, Y. Zhang, Y. Wang, Y. Yuan, G. Wang, and N. Tang. Lakebench: A benchmark for discovering joinable and unionable tables in data lakes. *Proceedings of the VLDB Endowment (PVLDB)*, 17(8):1925–1938, 2024.

[19] L. Ding, D. DiFranzo, A. Graves, J. Michaelis, X. Li, D. L. McGuinness, and J. Hendler. Data-gov wiki: Towards linking government data. In *Proceedings of the National Conference on Artificial Intelligence (AAAI)*. AAAI, 2010.

[20] Y. Dong, K. Takeoka, C. Xiao, and M. Oyamada. Efficient joinable table discovery in data lakes: A high-dimensional similarity-based approach. In *37th IEEE International Conference on Data Engineering, ICDE 2021, Chania, Greece, April 19-22, 2021*, pages 456–467. IEEE, 2021.

[21] Y. Dong, C. Xiao, T. Nozawa, M. Enomoto, and M. Oyamada. Deepjoin: Joinable table discovery with pre-trained language models. *Proceedings of the VLDB Endowment (PVLDB)*, 16(10):2458–2470, 2023.

[22] J. Eberius, M. Thiele, K. Braunschweig, and W. Lehner. Top-k entity augmentation using consistent set covering. SSDBM '15, 2015.

[23] M. Esmailoghli, J. Quiané-Ruiz, and Z. Abedjan. COCOA: correlation coefficient-aware data augmentation. In Y. Velegrakis, D. Zeinalipour-Yazti, P. K. Chrysanthis, and F. Guerra, editors, *Proceedings of the International Conference on Extending Database Technology (EDBT)*, pages 331–336. OpenProceedings.org, 2021.

[24] M. Esmailoghli, J. Quiané-Ruiz, and Z. Abedjan. MATE: multi-attribute table extraction. *Proceedings of the VLDB Endowment (PVLDB)*, 15(8):1684–1696, 2022.

[25] G. Fan, J. Wang, Y. Li, D. Zhang, and R. J. Miller. Semantics-aware dataset discovery from data lakes with contextualized column-based representation learning. *Proceedings of the VLDB Endowment (PVLDB)*, 16(7):1726–1739, 2023.

[26] R. C. Fernandez, Z. Abedjan, F. Koko, G. Yuan, S. Madden, and M. Stonebraker. Aurum: A data discovery system. In *Proceedings of the International Conference on Data Engineering (ICDE)*, pages 1001–1012. IEEE Computer Society, 2018.

[27] R. C. Fernandez, J. Min, D. Nava, and S. Madden. Lazo: A cardinality-based method for coupled estimation of jaccard similarity and containment. In *Proceedings of the International Conference on Data Engineering (ICDE)*, pages 1190–1201. IEEE, 2019.

[28] S. Galhotra, Y. Gong, and R. C. Fernandez. Metam: Goal-oriented data discovery. In *Proceedings of the International Conference on Data Engineering (ICDE)*, pages 2780–2793. IEEE, 2023.

[29] Y. Gu, J. Cao, S. Qian, N. Zhu, and W. Guan. MANSOR: A module alignment method based on neighbor information for scientific workflow. *Concurr. Comput. Pract. Exp.*, 36(10), 2024.

[30] A. Y. Halevy, F. Korn, N. F. Noy, C. Olston, N. Polyzotis, S. Roy, and S. E. Whang. Goods: Organizing google's datasets. In F. Özcan, G. Koutrika, and S. Madden, editors, *Proceedings of the International Conference on Management of Data (SIGMOD)*, pages 795–806. ACM, 2016.

[31] P. Holmes. Correlation: From picture to formula. *Teaching Statistics*, 23(3):67–71, 2001.

[32] M. Hong, M. Riedewald, C. Koch, J. Gehrke, and A. J. Demers. Rule-based multi-query optimization. In M. L. Kersten, B. Novikov, J. Teubner, V. Polutin, and S. Manegold, editors, *Proceedings of the International Conference on Extending Database Technology (EDBT)*, volume 360 of *ACM International Conference Proceeding Series*, pages 120–131. ACM, 2009.

[33] M. Hulsebos, Ç. Demiralp, and P. Groth. Gittables: A large-scale corpus of relational tables. *Proc. ACM Manag. Data*, 1(1):30:1–30:17, 2023.

[34] A. Khatiwada, G. Fan, R. Shraga, Z. Chen, W. Gatterbauer, R. J. Miller, and M. Riedewald. SANTOS: relationship-based semantic table union search. *Proceedings of the ACM on Management of Data (PACMMOD)*, 1(1):9:1–9:25, 2023.

[35] A. Khatiwada, R. Shraga, W. Gatterbauer, and R. J. Miller. Integrating data lake tables. *Proceedings of the VLDB Endowment (PVLDB)*, 16(4):932–945, 2022.

[36] O. Lehmberg and C. Bizer. Stitching web tables for improving matching quality. *Proceedings of the VLDB Endowment (PVLDB)*, 10(11):1502–1513, 2017.

[37] G. Li, J. He, D. Deng, and J. Li. Efficient similarity join and search on multi-attribute data. In *Proceedings of the International Conference on Management of Data (SIGMOD)*, pages 1137–1151. ACM, 2015.

[38] LinkedIn. WhereHows: A Data Discovery and Lineage Portal. https://github.com/linkedin/WhereHows, 2016.

[39] X. Ma, T. Teofili, and J. Lin. Anserini gets dense retrieval: Integration of lucene's HNSW indexes. In I. Frommholz, F. Hopfgartner, M. Lee, M. Oakes, M. Lalmas, M. Zhang, and R. L. T. Santos, editors, *Proceedings of the International Conference on Information and Knowledge Management (CIKM)*, pages 5366–5370. ACM, 2023.

[40] M. Mahdavi and Z. Abedjan. Baran: Effective error correction via a unified context representation and transfer learning. *Proceedings of the VLDB Endowment (PVLDB)*, 13(11):1948–1961, 2020.

[41] T. Milo and A. Somech. Automating exploratory data analysis via machine learning: An overview. In D. Maier, R. Pottinger, A. Doan, W. Tan, A. Alawini, and H. Q. Ngo, editors, *Proceedings of the International Conference on Management of Data (SIGMOD)*, pages 2617–2622. ACM, 2020.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:53 UTC from IEEE Xplore. Restrictions apply.

[42] M. Minor, A. Tartakovski, and R. Bergmann. Representation and structure-based similarity assessment for agile workflows. In R. Weber and M. M. Richter, editors, *Proceedings of the International Conference on Case-Based Reasoning, ICCBR*, volume 4626 of *Lecture Notes in Computer Science*, pages 224–238. Springer, 2007.

[43] R. Muthukrishnan and R. Rohini. Lasso: A feature selection technique in predictive modeling for machine learning. In *IEEE international conference on advances in computer applications (ICACA)*, pages 18–20. Ieee, 2016.

[44] F. Nargesian, K. Q. Pu, E. Zhu, B. G. Bashardoost, and R. J. Miller. Organizing data lakes for navigation. In D. Maier, R. Pottinger, A. Doan, W. Tan, A. Alawini, and H. Q. Ngo, editors, *Proceedings of the International Conference on Management of Data (SIGMOD)*, pages 1939–1950. ACM, 2020.

[45] F. Nargesian, E. Zhu, K. Q. Pu, and R. J. Miller. Table union search on open data. *Proceedings of the VLDB Endowment (PVLDB)*, 11(7):813–825, 2018.

[46] K. W. Ng, Z. Wang, R. R. Muntz, and S. Nittel. Dynamic query re-optimization. In Z. M. Özsoyoglu, G. Özsoyoglu, and W. Hou, editors, *Proceedings of the International Conference on Scientific and Statistical Database Management (SSDBM)*, pages 264–273. IEEE Computer Society, 1999.

[47] J. O'Neill and S. Dutta. Improved vector quantization for dense retrieval with contrastive distillation. In H. Chen, W. E. Duh, H. Huang, M. P. Kato, J. Mothe, and B. Poblete, editors, *Proceedings of the ACM Conference on Research and Development in Information Retrieval (SIGIR)*, pages 2072–2076. ACM, 2023.

[48] A. S. R. Santos, A. Bessa, F. Chirigati, C. Musco, and J. Freire. Correlation sketches for approximate join-correlation queries. In G. Li, Z. Li, S. Idreos, and D. Srivastava, editors, *Proceedings of the International Conference on Management of Data (SIGMOD)*, pages 1531–1544. ACM, 2021.

[49] A. S. R. Santos, A. Bessa, C. Musco, and J. Freire. A sketch-based index for correlated dataset search. In *Proceedings of the International Conference on Data Engineering (ICDE)*, pages 2928–2941. IEEE, 2022.

[50] S. Sarawagi and A. Kirpal. Efficient set joins on similarity predicates. In G. Weikum, A. C. König, and S. Deßloch, editors, *Proceedings of the International Conference on Management of Data (SIGMOD)*, pages 743–754. ACM, 2004.

[51] A. D. Sarma, L. Fang, N. Gupta, A. Y. Halevy, H. Lee, F. Wu, R. Xin, and C. Yu. Finding related tables. In K. S. Candan, Y. Chen, R. T. Snodgrass, L. Gravano, and A. Fuxman, editors, *Proceedings of the International Conference on Management of Data (SIGMOD)*, pages 817–828. ACM, 2012.

[52] T. K. Sellis. Multiple-query optimization. *ACM Trans. Database Syst.*, 13(1):23–52, 1988.

[53] K. Shim, T. K. Sellis, and D. S. Nau. Improvements on a heuristic algorithm for multiple-query optimization. *Data Knowl. Eng.*, 12(2):197–222, 1994.

[54] A. Simitsis, P. Vassiliadis, and T. K. Sellis. Optimizing ETL processes in data warehouses. In K. Aberer, M. J. Franklin, and S. Nishio, editors, *Proceedings of the International Conference on Data Engineering (ICDE)*, pages 564–575. IEEE Computer Society, 2005.

[55] A. Simitsis, P. Vassiliadis, and T. K. Sellis. State-space optimization of ETL workflows. *IEEE Trans. Knowl. Data Eng. (TKDE)*, 17(10):1404–1419, 2005.

[56] S. Suri, I. F. Ilyas, C. Ré, and T. Rekatsinas. Ember: No-code context enrichment via similarity-based keyless joins. *Proceedings of the VLDB Endowment (PVLDB)*, 15(3):699–712, 2021.

[57] Y. Tu, M. Eslami, Z. Xu, and H. Charkhgard. Multi-query optimization revisited: A full-query algebraic method. In S. Tsumoto, Y. Ohsawa, L. Chen, D. V. den Poel, X. Hu, Y. Motomura, T. Takagi, L. Wu, Y. Xie, A. Abe, and V. Raghavan, editors, *IEEE International Conference on Big Data*, pages 252–261. IEEE, 2022.

[58] B. F. van Dongen, R. M. Dijkman, and J. Mendling. Measuring similarity between business process models. In Z. Bellahsene and M. Léonard, editors, *Proceedings of Advanced Information Systems Engineering*, volume 5074 of *Lecture Notes in Computer Science*, pages 450–464. Springer, 2008.

[59] P. Venetis, Y. Sismanis, and B. Reinwald. *CRSI*: a compact randomized similarity index for set-valued features. In E. A. Rundensteiner, V. Markl, I. Manolescu, S. Amer-Yahia, F. Naumann, and I. Ari, editors, *Proceedings of the International Conference on Extending Database Technology (EDBT)*, pages 384–395. ACM, 2012.

[60] S. Wandelt, D. Deng, S. Gerdjikov, S. Mishra, P. Mitankin, M. Patil, E. Siragusa, A. Tiskin, W. Wang, J. Wang, and U. Leser. State-of-the-art in string similarity search and join. *Proceedings of the International Conference on Management of Data (SIGMOD)*, 43(1):64–76, 2014.

[61] L. Wenige, C. Stadler, M. Martin, R. Figura, R. Sauter, and C. W. Frank. Open data and the status quo - A fine-grained evaluation framework for open data quality and an analysis of open data portals in germany. *CoRR*, abs/2106.09590, 2021.

[62] C. Xiao, W. Wang, X. Lin, and H. Shang. Top-k set similarity joins. In Y. E. Ioannidis, D. L. Lee, and R. T. Ng, editors, *Proceedings of the 25th International Conference on Data Engineering, ICDE 2009, March 29 2009 - April 2 2009, Shanghai, China*, pages 916–927. IEEE Computer Society, 2009.

[63] C. Xiao, W. Wang, X. Lin, J. X. Yu, and G. Wang. Efficient similarity joins for near-duplicate detection. *ACM Trans. Database Syst.*, 36(3):15:1–15:41, 2011.

[64] M. Yakout, K. Ganjam, K. Chakrabarti, and S. Chaudhuri. Infogather: entity augmentation and attribute discovery by holistic matching with web tables. In K. S. Candan, Y. Chen, R. T. Snodgrass, L. Gravano, and A. Fuxman, editors, *Proceedings of the International Conference on Management of Data (SIGMOD)*, pages 97–108. ACM, 2012.

[65] M. Yu, G. Li, D. Deng, and J. Feng. String similarity search and join: a survey. *Frontiers Comput. Sci.*, 10(3):399–417, 2016.

[66] H. Zhang, A. S. R. Santos, and J. Freire. DSDD: domain-specific dataset discovery on the web. In G. Demartini, G. Zuccon, J. S. Culpepper, Z. Huang, and H. Tong, editors, *Proceedings of the International Conference on Information and Knowledge Management (CIKM)*, pages 2527–2536. ACM, 2021.

[67] Y. Zhang and Z. G. Ives. Juneau: Data lake management for jupyter. *Proceedings of the VLDB Endowment (PVLDB)*, 12(12):1902–1905, 2019.

[68] Y. Zhang and Z. G. Ives. Finding related tables in data lakes for interactive data science. In D. Maier, R. Pottinger, A. Doan, W. Tan, A. Alawini, and H. Q. Ngo, editors, *Proceedings of the International Conference on Management of Data (SIGMOD)*, pages 1951–1966. ACM, 2020.

[69] E. Zhu, D. Deng, F. Nargesian, and R. J. Miller. JOSIE: overlap set similarity search for finding joinable tables in data lakes. In P. A. Boncz, S. Manegold, A. Ailamaki, A. Deshpande, and T. Kraska, editors, *Proceedings of the International Conference on Management of Data (SIGMOD)*, pages 847–864. ACM, 2019.

[70] E. Zhu, F. Nargesian, K. Q. Pu, and R. J. Miller. LSH ensemble: Internet-scale domain search. *Proceedings of the VLDB Endowment (PVLDB)*, 9(12):1185–1196, 2016.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:53 UTC from IEEE Xplore. Restrictions apply.